

COMING TO TERMS WITH BUGS

Un bug può essere semplice come un errore di battitura in una formula o disastroso come un completo fraintendimento della definizione originale del problema.

Bug: un difetto in una soluzione che può causare comportamenti errati.

Dovreste abituarvi all'idea che le soluzioni che create non funzioneranno perfettamente. Conterranno bug da qualche parte, nonostante i vostri migliori sforzi. Anche i programmatori e gli scienziati informatici più talentuosi del mondo producono software che contiene bug,

I bug possono essere scoperti ed eliminati in qualsiasi fase del processo di sviluppo. Progettare per eliminare i bug è una buona pratica. Questo capitolo esaminerà tali pratiche.

Errore: qualunque comportamento osservato durante l'esecuzione di una soluzione che non corrisponde al comportamento atteso

Alcuni bug diventeranno evidenti solo dopo che la vostra soluzione è stata messa in atto. In questi casi, sono necessari approcci differenti, ovvero test e debug.

DESIGNING OUT THE BUGS

Errori di battitura

Nelle prime fasi di pianificazione il modo più semplice per prevenire errori in questa fase è cercare errori elementari per esempio:

- errori ortografici;
- errori grammaticali

Inconsistencies

Un'incoerenza si verifica quando parti diverse di una soluzione fanno affermazioni che non hanno senso se considerate insieme. Questi bug sono più difficili da localizzare perché ogni parte dell'incoerenza potrebbe apparire in contesti molto diversi.

Per esempio un tentativo di login fallito.

Se il login fallisce 5 volte, l'incidente dovrebbe essere segnalato alla sicurezza informatica e la seconda parte della soluzione, se il processo fallisce tre volte, la schermata di login dovrebbe bloccarsi in automatico.

Logical and mathematical errors

Bug semplici possono essere trovati nelle espressioni logiche.

Ad esempio l'algoritmo dovrebbe stampare il valore massimo tra due numeri, x e y . Se $x > y$, allora stampa y Altrimenti se $y > x$, allora stampa x

Questo contiene in realtà due bug: l'algoritmo stampa il minimo invece del massimo. Questo esempio. Errori matematici possono derivare anche da espressioni apparentemente valide dimostra che gli errori possono derivare da espressioni che sono valide

Being exhaustive

i computer non hanno buon senso; non cercheranno di colmare eventuali lacune nelle tue istruzioni e non possono condurre un dialogo con te per definire esattamente ciò che intendevi dire. gli algoritmi dovrebbero essere esaustivi e contenere istruzioni per tutti gli esiti concepibili. Se non prendi in considerazione alcuni di essi, il comportamento del computer diventa imprevedibile: potrebbe intraprendere un'azione indesiderata o potrebbe non fare nulla quando in realtà dovrebbe fare qualcosa. Questi tipi di errori sono facilmente commessi e sono comuni tra i principianti

General tips

Nel Capitolo 3 è stato raccomandato di affrontare la risoluzione dei problemi in modo iterativo. Bisogna fare del nostro meglio per mantenere le diverse parti delle nostre soluzioni autocontenute minimizzando il numero di relazioni tra di esse. Questo dovrebbe localizzare gli effetti di eventuali errori e aiutare a prevenire effetti a catena.

Infine se la soluzione del problema è stata fornita da qualcun altro, bisognerebbe chiedere a quella persona di verificare il progetto di soluzione. Gli equivoci tra il progettista della soluzione e il 'cliente' sono una fonte notoria di problemi.

MITIGATING ERRORS

la soluzione finita conterrà molto probabilmente dei bug, nonostante i migliori sforzi. La buona notizia è che esistono passi che si possono compiere contro gli errori.

Getting defensive:

Un modo per minimizzare gli effetti degli errori è anticipare dove potrebbe verificarsi un problema e poi mettere in atto delle 'barriere protettive'. L'informatica chiama questa programmazione difensiva. L'approccio verifica che certe condizioni siano come previsto prima di eseguire il compito principale della soluzione. Con un mancato rispetto di queste condizioni potrebbe verificarsi un potenziale fallimento del sistema.

Reacting to problems:

per capire come reagire ai problemi, bisognerebbe capire la gravità del problema stesso: Se la gravità è bassa, l'azione intrapresa potrebbe essere qualcosa di minore come mostrare un messaggio. Se la gravità è alta e rischia di causare danni reali, il sistema dovrebbe entrare in una sorta di modalità di emergenza o addirittura spegnersi. quando si reagisce agli errori includono:

se il sistema è interattivo o meno ;

se il sistema è transazionale o meno. Se lo è, allora è possibile annullare le modifiche effettuate durante un passaggio difettoso.

Checking user input:

Nei casi in cui la tua soluzione richiede un input da parte dell'utente, un po' di 'paranoia salutare' è molto utile. Gli umani commettono errori tutto il tempo quindi è meglio presumere che il tuo utente inserirà input errati in qualche momento.

Se la tua soluzione si aspetta un numero, verifica che sia realmente un numero prima di fare qualsiasi cosa con esso.

se la tua soluzione richiede un input entro un certo intervallo, controllalo dopo che è stato inserito

Formati errati: numeri di telefono, URL dei siti web, indirizzi email. Etc

Essere proattivi e proteggersi contro comportamenti indesiderati è essenziale.

TESTING:

Questa sezione descrive il testing, un metodo per localizzare i bug nascosti in un sistema completamente o parzialmente funzionante.

Goal:

L'obiettivo del testing è utilizzare il sistema reale che hai prodotto per verificare se funziona come previsto e per identificare aree specifiche in cui fallisce. il testing deve attendere fino a quando la tua soluzione non ha raggiunto un certo stato di funzionamento. Potrebbe essere completamente funzionante o potrebbe essere parzialmente funzionante, permettendoti di testare singole parti. Per eseguire efficacemente il testing, devi effettivamente sapere cosa mira a fare la soluzione

Approach:

l'approccio si può fare in 2 modi:

dall'alto al basso: si mette alla prova il desing per intero per vedere se funziona. Questo è più efficace per verificare se il sistema funziona completamente

dal basso verso l'alto: richiede di controllare parte per parte e vedere se il sistema ha delle fondamenta solide.

Testing individual parts:

I test unitari aiutano a localizzare i problemi, ma richiedono di simulare le dipendenze con dati fittizi. Invece di testare ogni valore possibile, si utilizzano classi di equivalenza con valori rappresentativi, rendendo il processo di testing più efficiente.

Per il gioco FizzBuzz, gli input vengono divisi in classi di equivalenza, e per ciascuna classe si sceglie un valore rappresentativo da testare. Le classi includono:

- **Numeri normali accettabili** (es. 8): numeri non divisibili per 3 o 5, la subroutine restituisce lo stesso numero.
- **Fizzes** (es. 3): numeri divisibili per 3, la subroutine restituisce "Fizz".
- **Buzzes** (es. 10): numeri divisibili per 5, la subroutine restituisce "Buzz".

- **FizzBuzzes** (es. 30): numeri divisibili per sia 3 che 5, la subroutine restituisce "FizzBuzz".
- **Numeri inaccettabili** (es. -10): numeri fuori dall'intervallo, devono essere rifiutati.
- **Non numeri** (es. VII): solo numeri arabi, i testi devono essere rifiutati.
- **Limite accettabile** (es. 1): deve funzionare.
- **Limite inaccettabile** (es. 0): deve essere rifiutato.

Il testing è utile per ottenere un'altra prospettiva. Dovresti registrare i test in un piano chiaro e comprensibile, poi farlo testare a una terza persona. Questo aiuta a individuare problemi che potresti non notare da solo.

DEBUGGING:

Durante il testing o l'uso della soluzione, potrebbero verificarsi errori enigmatici. In questi casi, è importante adottare un approccio sistematico per fare il debugging. Ispirandosi a Sherlock Holmes, l'obiettivo è esaminare i fatti (errori) e ricostruire la catena di eventi che li ha causati. Devi cercare indizi, fare deduzioni e costruire un'ipotesi che spieghi il problema, quindi testarla. Sebbene non esista un metodo infallibile per il debugging, con la pratica e l'uso di diverse strategie, diventerai più abile nel risolvere gli errori.

Be ruthless with hunches:

Se hai un'intuizione sul problema, provala. Conosci bene il sistema, quindi hai un'idea di come funziona e dove potrebbe esserci un errore. Tuttavia, se l'intuizione è sbagliata, abbandonala subito senza affezionarti troppo alla tua spiegazione.

Respect Occam's Razor:

Quando cerchi di risolvere un problema, considera prima le spiegazioni più semplici, che richiedono meno assunzioni. Il principio del Rasoio di Occam suggerisce che l'ipotesi più semplice è probabilmente quella corretta, quindi evita di complicare troppo le spiegazioni. Prova prima le soluzioni semplici ed escludi quelle più complesse.

Divide and conquer:

Se non sei sicuro di dove si trovi il problema, semplifica il processo utilizzando un approccio di **divide et impera**. Dividi il sistema in componenti separati e, se un problema si verifica prima che il controllo arrivi a un determinato componente, puoi escluderlo come causa. Inoltre, qualsiasi componente invocato solo da quello eliminato può essere escluso anch'esso.

Change one thing at a time:

Il buon debugging è simile a un esperimento scientifico: varia un solo fattore alla volta per identificare la causa di un problema. Se una modifica non risolve il problema, annullala e ripristina il sistema allo stato originale prima di provare altre soluzioni.

Logging:

Se noti un comportamento inaspettato nella tua soluzione, puoi indagare aggiungendo istruzioni di logging nel tuo algoritmo. Queste istruzioni segnaleranno quando determinati eventi accadono, come l'ingresso in un loop o l'esecuzione di una condizione. Ad esempio, se un loop dovrebbe essere eseguito cinque volte ma il log segnala solo quattro, significa che c'è un problema. Allo stesso modo, se una condizione `if` non viene attivata, puoi inserire un log per stampare il valore della variabile sospetta e verificare se è corretto, per poi indagare ulteriormente.

Tracing:

Per diagnosticare un problema, puoi esaminare il codice riga per riga, seguendo manualmente l'esecuzione e tenendo traccia dei cambiamenti nei dati. In alternativa, puoi usare un **debugger**, uno strumento che ti permette di eseguire il programma sotto il tuo controllo, monitorando le istruzioni e lo stato dei dati. Inoltre, spiegare il problema ad alta voce, sia da solo che a qualcun altro, può aiutarti a trovare nuove intuizioni e identificare errori semplici che potresti aver trascurato.

YOU CAN'T HAVE EVERYTHING: DECIDING WHICH ERRORS TO FIX:

Ogni software avrà bug, e l'importante è gestirli piuttosto che cercare la perfezione. Per farlo, è utile classificare i bug in base alla **gravità** (quanto impediscono l'uso corretto del sistema) e alla **priorità** (quanto urgentemente devono essere risolti). La priorità dipende da fattori come la **frequenza** del bug, il **valore della funzionalità** compromessa e il **tempo necessario per correggere** l'errore. Utilizzando una matrice di priorità, si possono ordinare i bug e decidere quali correggere prima, dando priorità a quelli con alta gravità e alta priorità, e affrontando quelli con bassa priorità e gravità alla fine.